

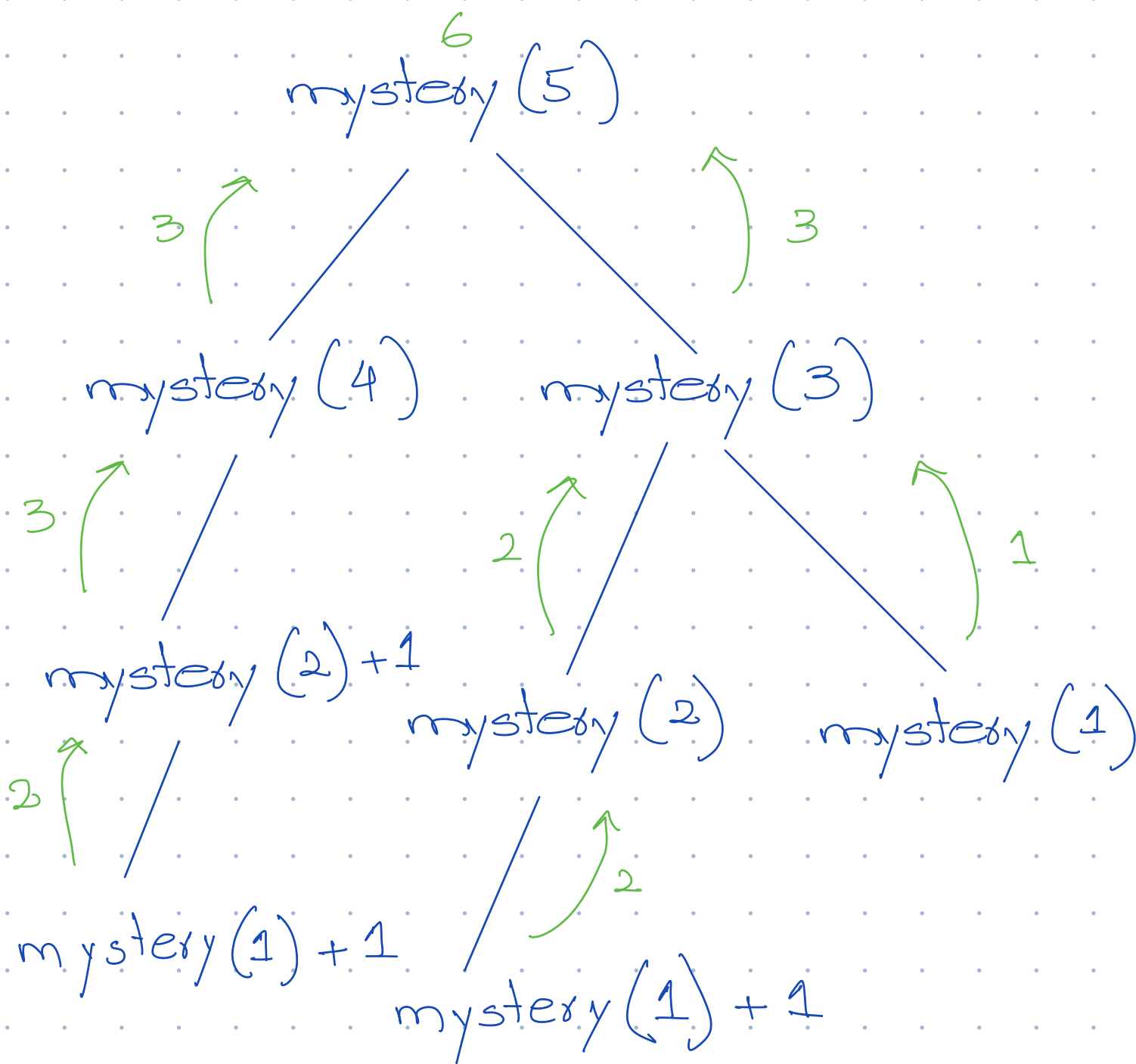
Lecture 06

Q1. mystery(5)

```
int mystery(int n) {  
    if (n <= 1) {  
        return 1;  
    }  
    if (n % 2 == 0) {  
        return mystery(n / 2) + 1;  
    }  
    return mystery(n - 1) + mystery(n - 2);  
}
```

composition / base case

decomposition



final return = 6

Time complexity = $O(2^n)$

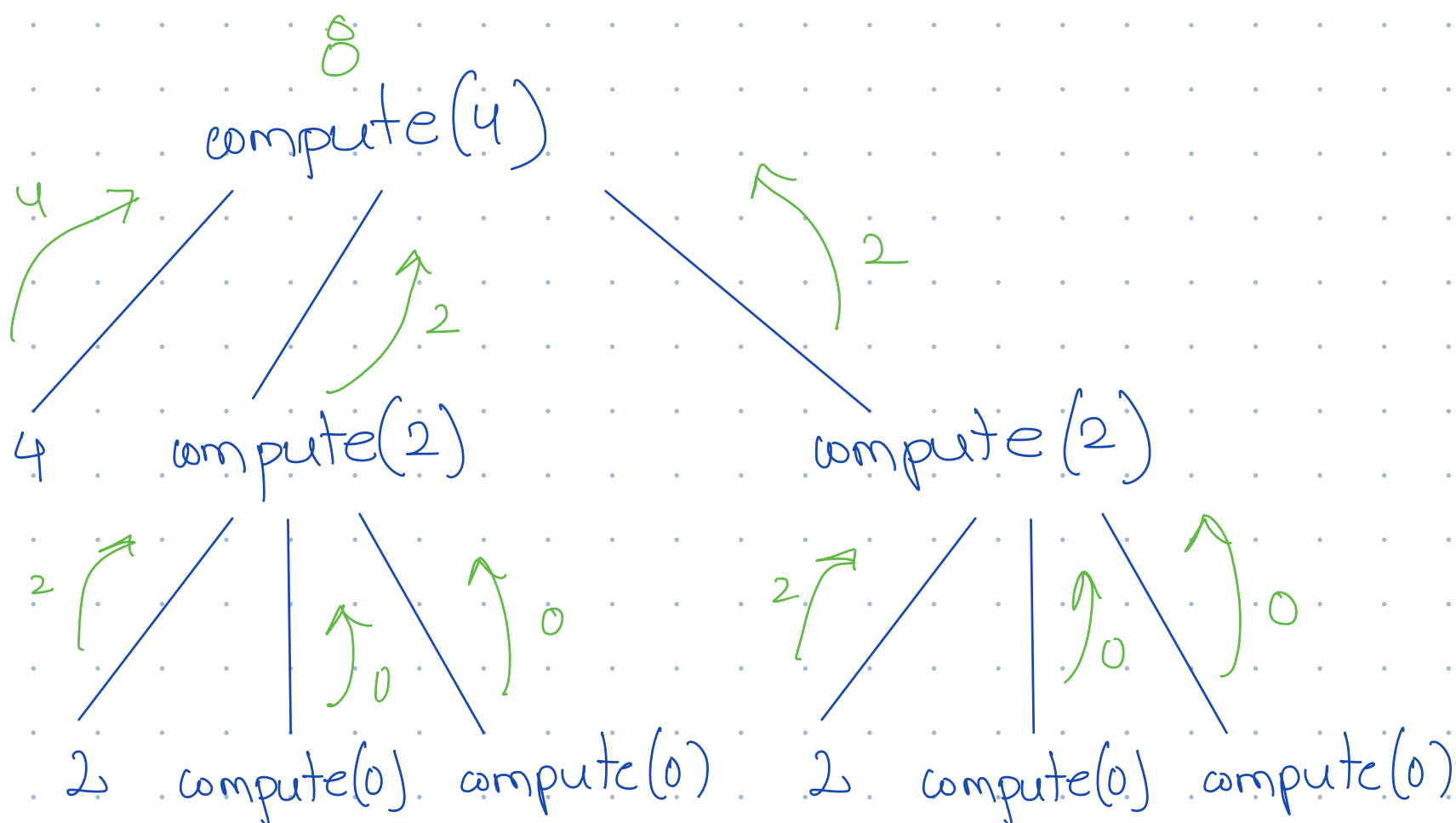
$$T(n) = 2T(n-1) + 1$$

The "Simplifying Assumption" says: Assume the code always takes the branch with the most calls, and assume those calls shrink the input as slowly as possible.

Q20

```
int compute(int n) {  
    if (n <= 0) {  
        return 0;  
    }  
    return n + compute(n - 2) + compute(n - 2);  
}
```

1.



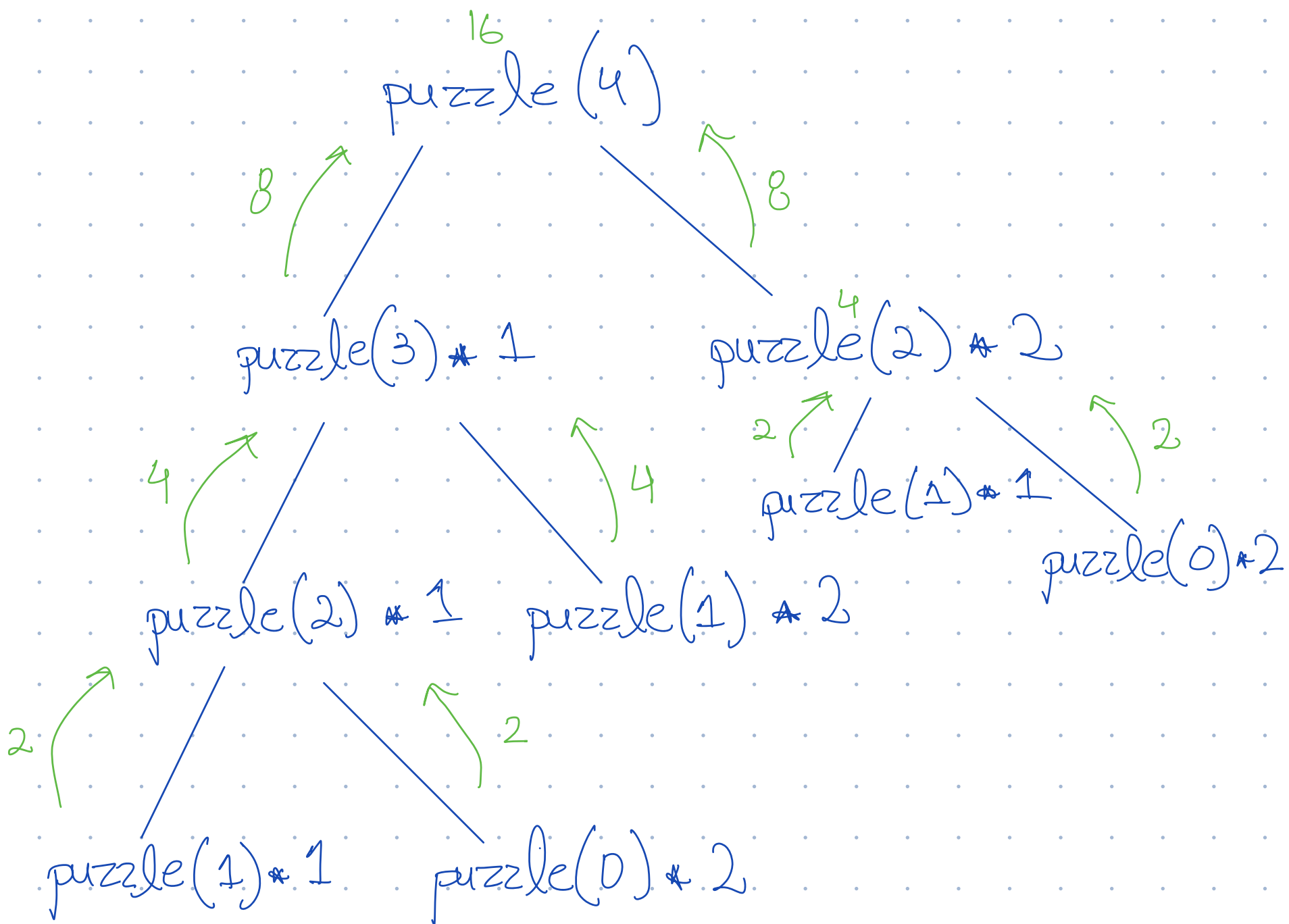
2. $T(n) = 2T(n-2) + 1$

3. $O(2^{n/2})$

Question 03

```
int puzzle(int n) {  
    if (n <= 0) return 1;  
    if (n == 1) return 2;  
  
    int sum = 0;  
    for (int i = 1; i <= 2; i++) {  
        sum += puzzle(n - i) * i;  
    }  
    return sum;  
}
```

10



$$2. \quad T(n) = \underbrace{2T(n-1)}_{O(2^n)} + 1$$

for simplicity we say $2T(n-1)$ as it will give an upper bound.

Question 04:

```
void process(int n) {
    if (n <= 1) return;

    process(n - 1);
    process(n - 3);
}
```

$$T(n) = 2T(n-1) + 1$$

$$O(2^n)$$

Q5.

```
int calc(int a, int b) {  
    if (b == 0) {  
        return 0;  
    }  
    if (b % 2 == 0) {  
        return calc(a + a, b / 2);  
    }  
    return calc(a + a, b / 2) + a;  
}
```

3. $T(n) = 1T(n/2) + 1 n^0 \log^0 n$

$$\log_2 \frac{1}{0} > 0$$

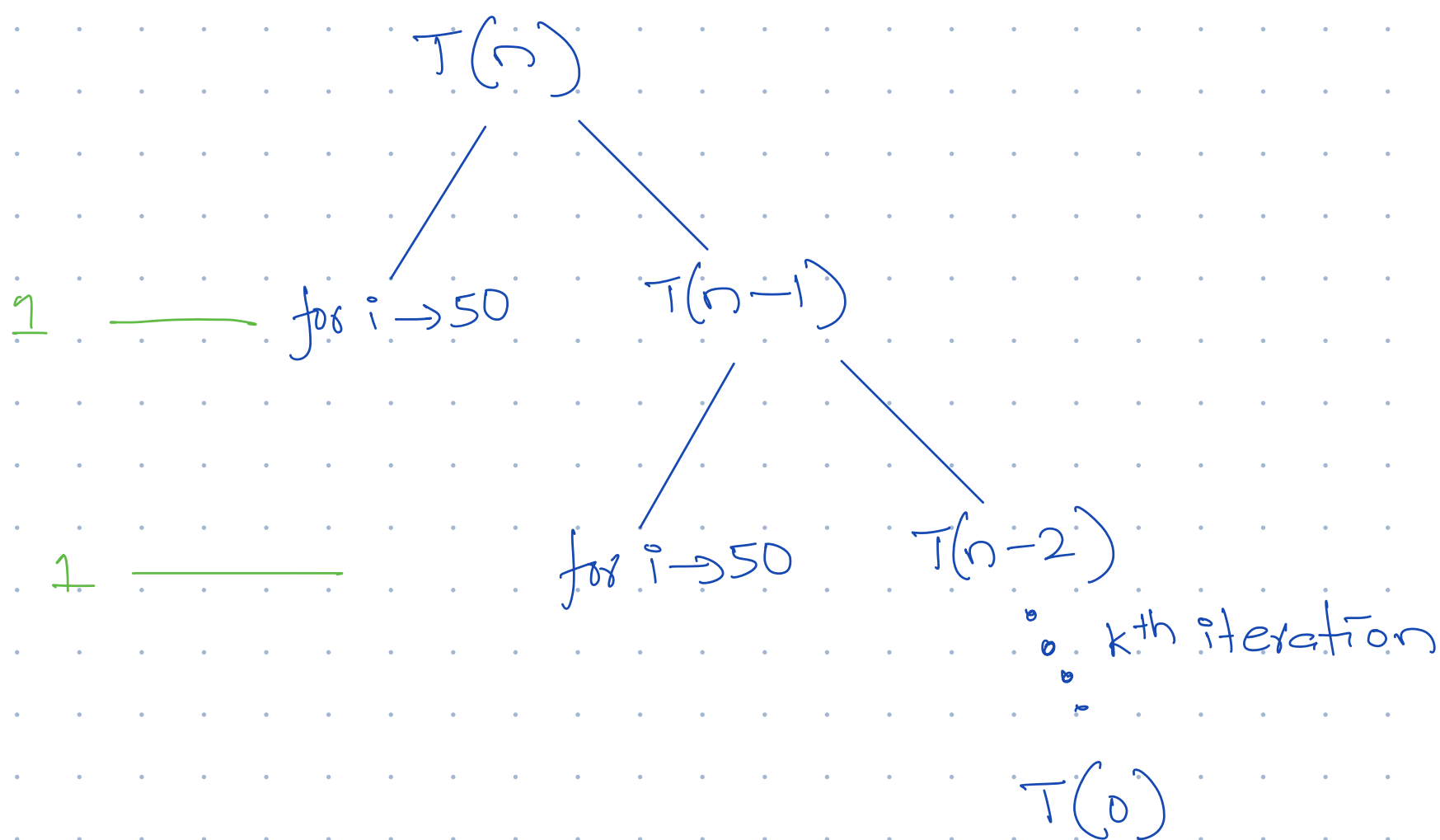
$$O(n^0 \log^{0+1} n)$$

$$O(\log n)$$

Lecture 07

Q1.

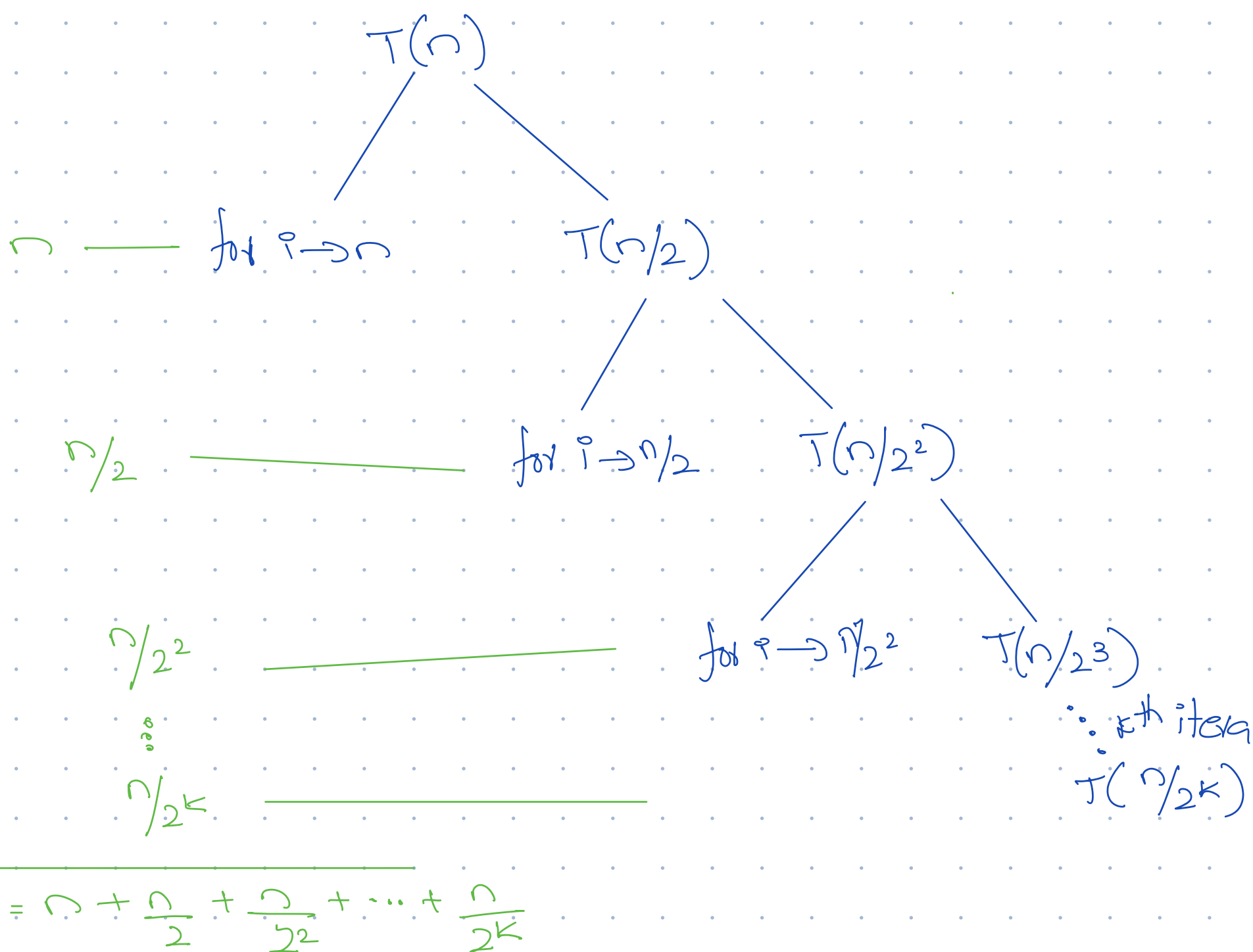
```
void processData(int n) { ————  $T(n)$   
    if (n <= 0) {  
        return;  
    }  
    for (int i = 0; i < 50; i++) {  
        printf("Processing\n");  
    }  
    processData(n - 1);  
}
```



$$= 1 + 1 + \dots$$
$$= n$$
$$O(n)$$

Q₂.

```
void divideTask(int n) {  $\text{--- } T(n)$ 
    if (n <= 1) {
        return;
    }
    for (int i = 0; i < n; i++) {
        printf("-");
    }
    divideTask(n / 2);
}
```



Terminate when

$$\frac{n}{2^k} = 1$$

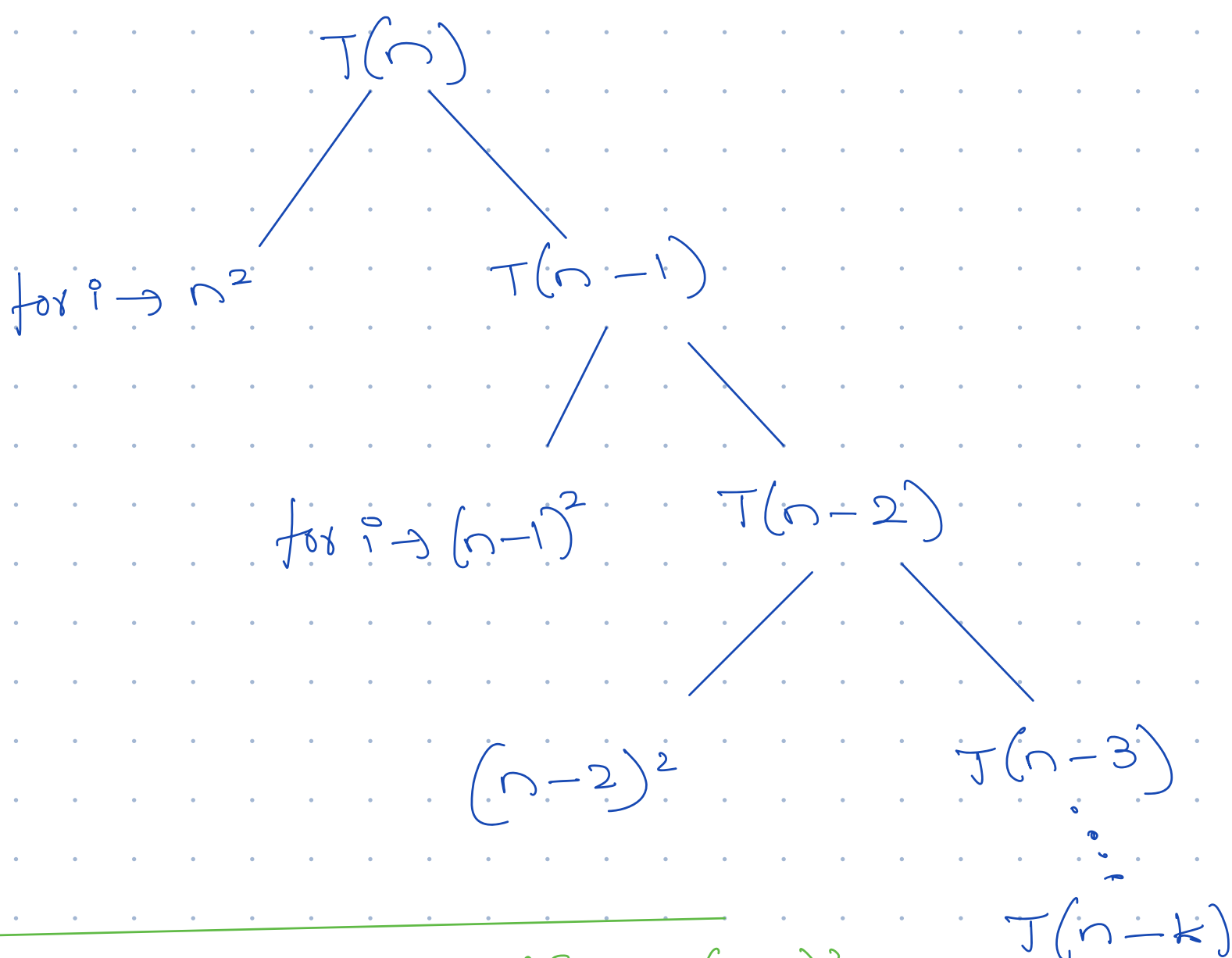
$$k = \log n$$

$$= n + \frac{n}{2} + \frac{n}{2^2} + \dots + 1 \rightarrow \text{geometric series}$$

$$= O(n)$$

Q3.

```
void conquerGrid(int n) { ———— T(n)
    if (n > 0) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                printf("*");
            }
        }
        conquerGrid(n - 1);
    }
}
```



$$= n^2 + (n-1)^2 + (n-2)^2 + \dots + (n-k)^2$$

Terminate when

$$n-k=0$$

$$n=k$$

$$= n^2 + (n-1)^2 + (n-2)^2 + \dots + 0 \rightarrow \text{sum of perfect squares series}$$

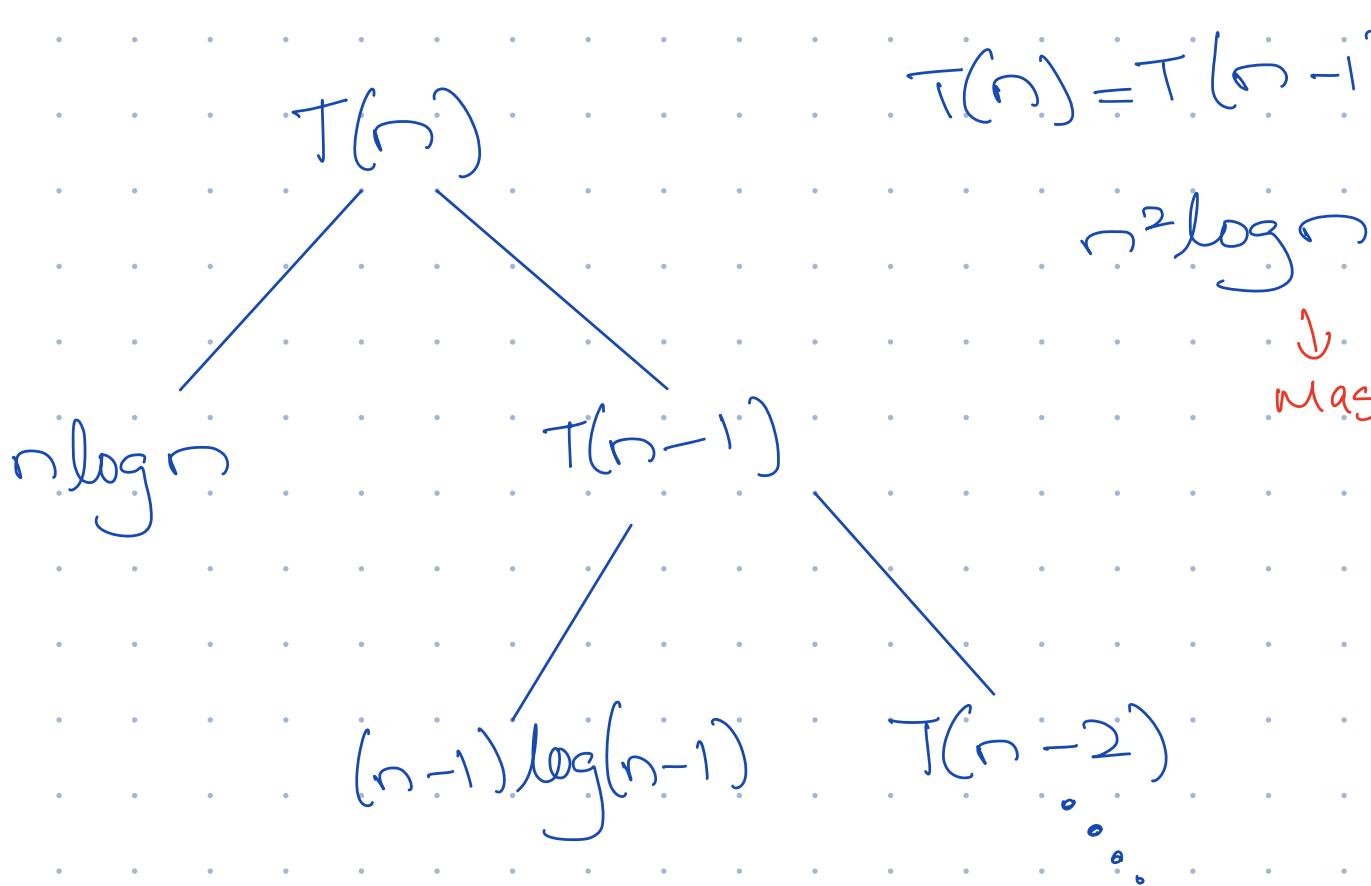
$$O(n^3)$$

Q4.

$$T(n) = T(n/2) + n \quad (?) \quad \text{(circled in yellow)}$$

Q5.

```
void complexTask(int n) { _____ T(n)
    if (n > 0) {
        for (int i = 0; i < n; i++) { _____ n
            for (int j = 1; j < n; j = j * 2) { _____ log n
                printf("Step\n");
            }
        }
        complexTask(n - 1);
    }
}
```



$$T(n) = T(n-1) + n \log n$$

$$n^2 \log n$$

↓
Master
theorem

$$n \log n + (n-1) \log(n-1) + \dots + (n-k) \log(n-k)$$

Terminate when

$$n-k=0$$

$$k=n$$

$$n \log n + (n-1) \log(n-1) + \dots + 0$$

$$O(n^2 \log n)$$

Lecture 8-9

Q1.

```
int trickyRecursion(int n) {  
    if (n <= 0) return 0;  
  
    int cost = 0;  
    for (int i = 0; i < n; i++) {  
        cost++;  
    }  
  
    return trickyRecursion(n + 2) - cost;  
}
```

→ Couldn't have made it more obvious.
Just give me the answer
next time.

You have subtract 2 for it to be
a valid recursive function

Q2.

```
void processSkips(int n) {  
    if (n <= 0) return;  
  
    for (int i = 0; i < n; i++) {  
        printf("Processing\n");  
    }  
  
    processSkips(n - 2);  
}
```

$$T(n) = T(n-2) + n \text{ --- M}$$

put $n = n-2$ in M

$$T(n-2) = T(n-4) + n + (n-2)$$

put $T(n-2)$ in M

$$T(n) = T(n-4) + (n-2) + n \text{ --- W}$$

put $n = n-4$ in M

$$T(n-4) = T(n-6) + (n-4)$$

put $T(n-4)$ in i

$$T(n) = T(n-6) + (n-4) + (n-2) + n$$

$$T(n) = T(n-2k) + (n-2k) + \dots + (n-4) + (n-2) + n$$

Terminate when

$$n-2k=0$$

$$k = \frac{n}{2}$$

$$T(n) = T(n-2(n/2)) + (n-2(n/2)) + \dots + (n-4) + (n-2) + n$$

$$= T(0) + 0 + \dots + (n-4) + (n-2) + n$$

$$= 1 + 0 + \dots + (n-4) + (n-2) + n$$

$$O(n^2)$$

Q3.

```
void analyzeLog(int n) {  
    if (n <= 1) return;  
  
    for (int i = 1; i < n; i = i * 2) {  
        printf("Log step\n");  
    }  
  
    analyzeLog(n - 3);  
}
```

$$T(n) = T(n-3) + \log n \quad \text{--- M}$$

put $n = n-3$ in M

$$T(n-3) = T(n-6) + \log(n-3)$$

put $T(n-3)$ in M

$$T(n) = T(n-6) + \log(n-3) + \log n$$

$$T(n) = T(n-3k) + \log(n-3k) + \dots + \log(n-3) + \log n$$

Terminate when

$$n-3k=1$$

$$n-1=3k$$

$$k = \frac{n-1}{3}$$

$$T(n) = T\left(n-3\left(\frac{n-1}{3}\right)\right) + \log\left(n-n\left(\frac{n-1}{3}\right)\right) + \dots + \log(n)$$

$$= T(1) + \log(1) + \dots + \log(n-6) + \log(n)$$

$$= 1 + 0 + \dots + \log(n-6) + \log(n)$$

$$= 1 + \sum_{i=0}^{\frac{n}{3}-1} \log(n-3i)$$

$$= 1 + \frac{n}{3} \log n$$

$$O(n \log n)$$

Q4.

This NOT in syllabus for Quiz.

I told to ONLY focus on concepts in the slides.

Dekho Zayan bhai apke GPT kiya ker raha (??)

Q5.

```
int complexIter(int n) {
    if (n <= 0) return 0;

    int step = 0;
    for (int i = 0; i < n; i++) {        n
        for (int j = 0; j < 3; j++) {        3
            step++;
        }
    }
}
```

```
return complexIter(n - 1) + step;
```

```
}
```

$$T(n) = T(n-1) + \underbrace{n}_{\text{dropping 3 as it's a constant}}$$

$$O(n^2)$$

Intentionally left
blank *

Overall practice

Q1.

```
void algorithmA(int n) {  
    if (n <= 1) {  
        return;  
    }  
    for (int i = 0; i < n; i++) {  
        printf("*");  
    }  
    algorithmA(n / 2);  
    algorithmA(n / 2);  
}
```

$$T(n) = 2T(n/2) + n \quad \text{--- (i)}$$

put $n = n/2$ in (i)

$$T(n/2) = 2T(n/2^2) + \frac{n}{2}$$

put $T(n/2)$ in (i)

$$\begin{aligned} T(n) &= 2 \left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \right] + n \\ &= 2^2 T\left(\frac{n}{2^2}\right) + n + n \quad \text{--- (ii)} \end{aligned}$$

put $n = \frac{n}{2^2}$ in (ii)

$$T(n/2^2) = 2^2 T(n/2^3) + \frac{n}{2^2}$$

put $T(n/2^2)$ in (ii)

$$\begin{aligned} T(n) &= 2^2 \left[2T(n/2^3) + \frac{n}{2^2} \right] + n + n \\ &= 2^3 T(n/2^3) + 3n \end{aligned}$$

• kth iteration
•

$$T(n) = 2^k T(n/2^k) + kn$$

Terminate when

$$\frac{n}{2^k} \leq 1$$

$$k \leq \log n$$

$$T(n) = 2^{\log_2 n} T\left(\frac{n}{2^{\log_2 n}}\right) + \log n \cdot n$$

$$= n + n \log n$$

$$O(n \log n)$$

Q3.

```
void algorithmB(int n) {
    if (n <= 1) {
        return;
    }
    for (int i = 0; i < n; i++) {
        printf("-");
    }
    algorithmB(n / 3);
    algorithmB(n / 3);
    algorithmB(n / 3);
}
```

$$T(n) = 3T(n/3) + n \text{ --- M}$$

put $n = n/3$ in M

$$T(n/3) = 3T(n/3^2) + \frac{n}{3}$$

put $T(n/3)$ in M

$$T(n) = 3 \left[3T(n/3^2) + \frac{n}{3} \right] + n$$

$$= 3^2 T(n/3^2) + 2n$$

... kth iter

$$T(n) = 3^k T\left(\frac{n}{3^k}\right) + kn$$

Terminate when

$$\frac{n}{3^k} \leq 1$$

$$n \leq 3^k$$

$$\log_3 n \leq \log_3(3^k)$$

$$\log_3 n \leq k$$

$$T(n) = nT(1) + \log_3 n \cdot n$$

$$O(n \log_3 n)$$

Q3.

```
void algorithmC(int n) {
    if (n <= 1) {
        return;
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int temp = i + j;
        }
    }
    algorithmC(n / 2);
    algorithmC(n / 2);
    algorithmC(n / 2);
    algorithmC(n / 2);
}
```

$$T(n) = 4T(n/2) + n^2 \text{ --- M}$$

put $n = n/2$ in M

$$T(n/2) = 4T(n/2^2) + \frac{n^2}{4}$$

put $T(n/2)$ in M

$$T(n) = 4 \left[4T\left(\frac{n}{2^2}\right) + \frac{n^2}{4} \right] + n^2$$

$$= 4^2 T\left(\frac{n}{2^2}\right) + 2n^2$$

kth

$$T(n) = 4^k T\left(\frac{n}{2^k}\right) + kn^2$$

Terminate when

$$\frac{n}{2^k} \leq 1$$

$$k \leq \log n$$

$$\begin{aligned}
 T(n) &= 2 \log n T\left(\frac{n}{2 \log n}\right) + \log n \cdot n^2 \\
 &= n^2 T(1) + n^2 \log n \\
 &O(n^2 \log n)
 \end{aligned}$$

Q4.

```

void algorithmD(int n) {
    if (n <= 2) {
        return;
    }
    for (int i = 0; i < n; i++) {
        printf("Step\n");
    }
    algorithmD(n / 3);
    algorithmD(2 * n / 3);
}

```

$$\frac{2}{3} > \frac{1}{3}$$

So, $\frac{2n}{3}$ will converge slower as it will have a larger chunk of n each call. So, $T(n/3) + T(2n/3) \leq 2T(2n/3)$

$$\text{So, } T(n) = 2T\left(\frac{2n}{3}\right) + n \text{ --- M}$$

$$b = 3/2, a = 2, k = 1, p = 0$$

$$\log_{3/2} 2 > 1$$

$$O(n^{\log_{3/2} 2})$$

$$\text{put } n = \frac{2n}{3} \text{ in M}$$

$$T(2n/3) = 2T\left(\frac{2^2 n}{3^2}\right) + \frac{2n}{3}$$

$$\text{put } T(2n/3) \text{ in M}$$

$$T(n) = 2 \left[2T\left(\frac{2^2 n}{3^2}\right) + \frac{2n}{3} \right] + n$$

$$= 2^2 T\left(\frac{2^2 n}{3^2}\right) + \frac{2^2 n}{3} + n \text{ --- S}$$

$$\text{put } n = \frac{2^2 n}{3^2} \text{ in M}$$

$$T\left(\frac{2^2 n}{3^2}\right) = 2T\left(\frac{2^3 n}{3^2}\right) + \frac{2^2 n}{3}$$

put $T\left(\frac{2^2 n}{3^2}\right)$ in M

$$T(n) = 2^2 \left[2T\left(\frac{2^3 n}{3^3}\right) + \frac{2^2 n}{3^2} \right] + \frac{2^2 n}{3} + n$$

$$= 2^3 T\left(\frac{2^3 n}{3^3}\right) + \left(\frac{4}{3}\right)^2 n + \frac{4n}{3} + n$$

$$T(n) = 2^k T\left(\frac{2^k n}{3^k}\right) + \left(\frac{4}{3}\right)^{k-1} n + \dots + \left(\frac{4}{3}\right)^2 n + \frac{4n}{3} + n$$

Terminate when

$$\frac{2^k n}{3^k} \leq 2$$

$$\left(\frac{2}{3}\right)^k n \leq 2$$

$$n \leq 2 \cdot \left(\frac{3}{2}\right)^k$$

$$\log_{3/2} n \leq \log_{3/2}(2) + k \log_{3/2}\left(\frac{3}{2}\right)$$

$$\log_{3/2} n \leq k$$

geometric sum

$$T(n) = 2^{\log_{3/2} n} T\left(\frac{n}{\left(\frac{3}{2}\right)^{\log_{3/2} n}}\right) + \left(\frac{4}{3}\right)^{\log_{3/2} n} - 1 + \dots + n$$

$$= 2^{\log_{3/2} n} T(1) + 3n \left(n^{\log_{3/2}(4/3)} - 1 \right)$$

$$O(n^{\log_{3/2}(2)})$$

put $n = 2 \cdot 3^k$ in M
 $T(n) = 2^2 \left[2T\left(\frac{2^3 n}{3^3}\right) + \frac{2^2 n}{3^2} \right] + \frac{2^2 n}{3} + n$
 $T(n) = 2^3 T\left(\frac{2^3 n}{3^3}\right) + \left(\frac{4}{3}\right)^2 n + \frac{4n}{3} + n$
 $\vdots$
 $T(n) = 2^k T\left(\frac{2^k n}{3^k}\right) + \left(\frac{4}{3}\right)^{k-1} n + \dots + \frac{4n}{3} + n$
 $\vdots$
 $T(n) = 2^{\log_{3/2} n} T(1) + 3n \left(n^{\log_{3/2}(4/3)} - 1 \right)$
 $\vdots$
 $O(n^{\log_{3/2}(2)})$

Q5.

```
void algorithmE(int n) {  
    if (n <= 1) {  
        return;  
    }  
    for (int i = 1; i < n; i = i * 2) {  
        printf("+");  
    }  
    algorithmE(n / 2);  
    algorithmE(n / 2);  
}
```

$$T(n) = 2T(n/2) + \log_2 n$$

put $n = \frac{n}{2}$ in it

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{2^2}\right) + \log_2\left(\frac{n}{2}\right)$$

put $T(n/2)$ in it

$$T(n) = 2 \left[2T\left(\frac{n}{2^2}\right) + \log_2\left(\frac{n}{2}\right) \right] + \log_2 n$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + 2 \log_2 \frac{n}{2} + \log_2 n$$

$$= 2^k T\left(\frac{n}{2^k}\right) + 2^k \log_2 \frac{n}{2^k}$$

Terminate when

$$k \leq \log_2 n$$

$$T(n) = 2^{\log_2 n} T\left(\frac{n}{2^{\log_2 n}}\right) + \cancel{2^{\log_2 n}} \log_2 \left(\frac{\cancel{n}}{\cancel{2^{\log_2 n}}}\right)$$

$$= n T(1) + n(0)$$

$$O(n)$$

Game play:

Q1.

```
void house1(int n) {  
    if (n <= 1) return;  
  
    for (int i = 1; i < n; i++) {  
        for (int j = 1; j < n; j = j * 3) {  
            printf("valuable\n");  
        }  
    }  
  
    house1(n / 2);  
    house1(n / 2);  
    house1(n / 2);  
}
```

$\log_3 n$

$$T(n) = 3T(n/2) + n \log_3 n$$

Q2.

```
void house2(int n) {  
    if (n <= 0) return;  
  
    for (int i = n; i > 0; i = i / 2) {  
        printf("jewel\n");  
    }  
  
    house2(n - 2);  
    house2(n - 2);  
}
```

$$T(n) = 2T(n-2) + \log_2 n$$

Q3.

```
void house3(int n) {  
    if (n <= 4) return;  
  
    for (int i = 0; i < n * n; i++) {  
        printf("cash\n");  
    }  
  
    house3(n / 4);  
}
```



```
house3(3 * n / 4);
```

```
}
```

$$T(n) = 2T\left(\frac{3n}{4}\right) + n^2$$

Q4.

```
void house4(int n) {
    if (n <= 1) return;
```

```
    int j = 1;
    while (j < n) {
        j = j + j;
    }
```

```
    house4(n / 2);
    house4(n / 2);
    house4(n / 2);
    house4(n / 2);
}
```

j

1	—	2^0
2	—	2^1
4	—	2^2
8	—	2^3

...

Terminate when
 $2^k < n$

$$T(n) = 4T(n/2) + \log n$$

Q5.

```
void house5(int n) {
    if (n <= 2) return;
```

```
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("safe\n");
        }
    }
```

```
    house5(n / 3);
    house5(n / 3);
}
```

0
1
2
3
...

0
1
2
3
...

1+2+...+k
Terminate when
 $k \leq n$

$$\frac{1+2+\dots+n}{2} = \frac{n(n+1)}{2}$$

$$T(n) = 2T(n/3) + n^2$$

i
0
1
2
⋮
k

j
0 → n → n
1 → n → n-1
2 → n → n-2

k → n → n-k

$$n + (n-1) + (n-2) + \dots + (n-k)$$

Terminate when

$$n-k < 0$$

$$n \leq k$$

$$n + (n-1) + (n-2) + \dots + 0$$

$$\frac{n(n+1)}{2}$$

i
1
2
3
4
⋮
k²

i
1
2
3
⋮
k

j
1 → n
1, 3, 5, 7, ..., n
1, 4, 7, 10, 13, 16, ..., n
⋮

$$1, 1+k, 1+k+k, 1+k+k+k, \dots, n$$

$$1 + k \cdot n$$

Terminate when

i
1
2
3
⋮
k

p
0+1
0+1+2
0+1+2+3
⋮
0+1+2+...+k

stop when

$$\frac{k(k+1)}{2}$$

$$k^2 \leq n$$

$$\sqrt{n}$$

$$1, 2, 4, 16, 256,$$

$$2^1, 2^2, 2^4$$

$$2^{2^0}, 2^{2^1}, 2^{2^2}, 2^{2^3}, \dots, 2^{2^k}$$

$$2^{2^k} \leq n$$

$$2^k \log_2(2) \leq \log(n)$$

$$2^k \leq \log(n)$$

$$k \leq \log(\log n)$$

$$1, 1, 2, 4, \dots, 2^k$$

$$1, 1, 2, 4, \dots, 2^k$$

$$\frac{1 + 2 + 3 + \dots + 2^k}{1 + 2 + \dots + 2^{\log n}}$$

$$1, 1, 2, 3, \dots, k$$

$$1, \frac{n}{2}, \frac{n}{3}, \dots, \frac{n}{k}$$

$$\frac{n + \frac{n}{2} + \frac{n}{3} + \dots + \frac{n}{k}}{n}$$

$$n \left(1 + \frac{1}{2} + \dots + \frac{1}{k} \right)$$

$$n \log n$$

$$\frac{n}{k} \leq 1$$

$$n \leq k$$

```
int k = 1;
for (int i = 0; i < n; i += k) {
    k++;
    printf("A");
}
```

i	k
0	1
2	2
5	3
9	4
...	...
2	n

$$0 + 2 + 5 + 9 + \dots + 2$$

$$2^2 < n$$

$$2 < \sqrt{n}$$

$$O(\sqrt{n})$$

i	j
0	0
1	1
2	4
3	9
4	16
⋮	
k	k ²

$$0 + 1 + 4 + 9 + 16 + \dots + k^2$$

Terminate when

$$k^2 \leq n^2$$

$$0 + 1 + 4 + 9 + 16 + \dots + n^2 = n^3$$

$$O(n^4)$$

i	j
2 ⁰	1
2 ¹	(2 ¹) ²
2 ²	(2 ²) ²
⋮	⋮
2 ^k	(2 ^k) ²

$$2^{2k} \leq n^2$$

$$2^k \leq n$$

$$\log(n)$$

$$1 + 4 + 16 + \dots + 4^{\log n}$$